

GPU-based Parallel R-tree Construction and Querying

Sushil K. Prasad, Michael McDermott, Xi He, and Satish Puri

Department of Computer Science

Georgia State University

Atlanta, GA, USA

Email: sprasad@gsu.edu, {mmcdermott2, xhe8, spuri2}@student.gsu.edu

Abstract—An R-tree is a data structure for organizing and querying multi-dimensional non-uniform and overlapping data. Efficient parallelization of R-tree is an important problem due to societal applications such as geographic information systems (GIS), spatial database management systems, and VLSI layout which employ R-trees for spatial analysis tasks such as map-overlay. As graphics processing units (GPUs) have emerged as powerful computing platforms, these R-tree related applications demand efficient R-tree construction and search algorithms on GPUs. This problem is hard both due to (i) non-linear tree topology of the data structure itself and (ii) the unconventional single-instruction multiple-thread (SIMT) architecture of modern GPUs requiring careful engineering of a host of issues. Therefore, the current best parallelizations of R-tree on GPU has limited speedup of only about 20-fold. We present a space-efficient data structure design and a non-trivial bottom-up construction algorithm for R-tree on GPUs. This has yielded the first demonstrated 226-fold speedup in parallel construction of an R-tree on a GPU compared to one-core execution on a CPU. We also present innovative R-tree search algorithms that are designed to overcome GPU's architectural and resource limitations. The best of these algorithms gives a speed up of 91-fold to 180-fold on an R-tree with 16384 base objects for query sizes ranging from 2k to 16k.¹

Keywords—R-Tree, GPGPU, Recursion, Hierarchical Data Structure, Dynamic Parallelism, GIS, CUDA

I. INTRODUCTION

The R-tree is an important data structure for multi-dimensional data. It can be used, for example, for range queries which retrieve all records whose attributes represent d -dimensional boxes located in a given d -dimensional query box. Without a loss of generality, this work uses d equal to 2. An R-tree is an hierarchical data structure. Each R-tree node contains a certain number of index entries, each of which consists of a minimum bounding rectangle (MBR) and pointer to a spatial object if it is a leaf node, or a pointer to its child node if it is a non-leaf node. The MBR is a critical meta-data for building an R-tree. Each spatial object can be represented by its MBR while each index entry in a non-leaf node can also be represented by a MBR which contains the MBRs of all spatial objects or index entries in its children nodes. Figure

1 illustrates the layout of MBRs of spatial object's and R-tree node's index entries. Here, A through C represent three index entries in the root node while D through N are spatial objects being indexed by the data structure. Figure 2 is the corresponding R-tree with four nodes. R1 is the parent node of R2, R3, and R4.

Fig. 1. A layout of MBRs

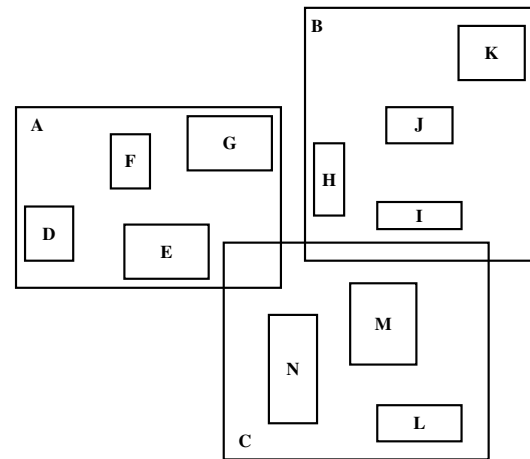
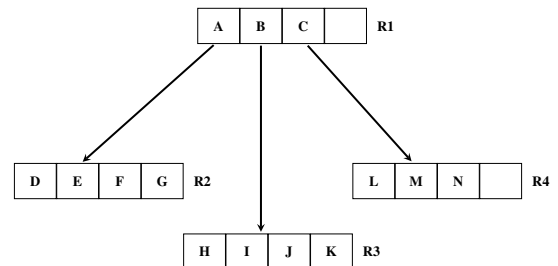


Fig. 2. A two-level R-tree representing the MBRs



As GPUs are widely used in both industry and academics, research into parallel data structures on many-core architectures has become urgent and promising. A number of tree-based data structures, such as R-tree [1], KD-tree [2–4], Octree [5–11], decision tree [12], [13], and bounding volume

¹Project page with additional information and ongoing work can be found at <http://www.cs.gsu.edu/~dimos/?q=GPGPU-RTree>

hierarchy [14] have been ported to GPGPUs. Besides the tree-based data structures, hash tables on GPGPUs have also been discussed in [15–17]. The difficulty of efficiently porting data structures to GPUs lies in the non-trivial implementation of data structures' operations so that GPUs' streaming processors can be fully utilized and a good speedup can be achieved. GPUs' SIMT architecture, latency toleration mechanism, and inefficient synchronization are important factors to be considered. A few existing parallelizations of R-tree on GPUs [1], [18] are limited, resulting in poor quality R-tree construction and inefficient memory usage and allocation (Section II).

Main contributions in this paper can be summarized as follows:

- We have designed and developed a parallel R-tree construction algorithm for GPUs. A potentially transformative concept in the proposed algorithms is *Geo-Packing principle*. On GPUs, the spatially co-related objects can be exploited both for efficient parallel construction of R-tree data structure via their bottom-up packing, and for efficient parallel search of R-tree by their top-down packing. Such Geo-Packing principle can harness GPU-specific memory coalescing and synchronous thread grouping (warp). We demonstrate more than 200-fold speedup for R-tree construction on a nVidia Tesla K20 as compared to a single core Intel Xeon E5-2650 v2 CPU performance.
- We have designed and developed several parallel R-tree query algorithms for the GPU. They range from an optimized exhaustive search of the leaf level query space to a level by level top-down search, with combination of BFS and DFS techniques. The best of these algorithms offers speedups in the range of 91-fold to 180-fold on an R-tree with 16384 base objects for query sizes ranging from 2048 to 16384 on an nVidia Tesla K20.

Currently our R-Tree construction implementation is being used on our CPU-GPU Parallel Polygon Overlay System [19].

The paper's organization is as follows: Section II briefly reviews the related work. Section III gives an overview of the nVidia architecture and CUDA. Section IV presents the parallel construction algorithm for R-tree data structure in CUDA. Section V describes the R-tree search algorithms developed in CUDA. In Section VI, comprehensive experiments conducted to evaluate the system performance are described. We offer some conclusions and future road map in Section VII.

II. RELATED WORK

A. Parallel R-tree Construction

In the seminal R-tree paper [20], an R-tree is built by inserting new items iteratively as they arrive. Each insert operation of a polygonal object represented by its Minimum Bounding Rectangle (MBR) first follows one path from root down to a leaf such that all nodes along this branch contain overlapping MBRs. Then the new item is added to the leaf node. A similar algorithm is employed for spatial search requiring all overlapping branches to be visited. If the leaf node does not have enough space for the new item, it will be split into two nodes, which will insert a new item into its

parent node. Such change might recursively propagate upward along the same path until the root node. If the root node is full, it will be split into two new nodes and a new root will be created, like B-tree [21], its counterpart for one dimensional data. R-tree is designed for accelerating search performance for bulk data. The way an R-tree is built plays a crucial role in determining the search performance. In general, two primary goals are considered during the construction of R-tree. The first goal is to minimize the area of the MBRs of the non-leaf nodes that are not covered by MBRs at the leaves these enclose, which is also called the "dead area." This goal can improve search performance since decisions on which paths have to be traversed can be taken on higher levels. The second goal is to minimize the overlap between MBRs. This goal is also designed to decrease the number of paths to be traversed. Some R-tree variations, such as R* tree [22] and Hilbert R-tree [23], are designed towards achieving these two goals by employing different heuristic-based splitting algorithms. R⁺ tree, another R-tree variation, may partition a to-be-inserted rectangle-bounded spatial object into two spatial objects and avoids possible overlap between MBRs on the same level. These R-tree construction methods are inherently sequential because the insert operations need to compete to access R-tree nodes, and the node splitting operation may require locking of the whole R-tree and prevent concurrent insertions. However, in case of some classes of applications in which all the items are available *a priori* such as the GIS problems, R-tree can also be built by packing techniques. Existing R-tree packing technique can be classified into two categories: bottom-up packing [24] and top-down packing [25], [26]. As their names imply, bottom-up packing builds an R-tree from leaf-level to root-level by merging, while top-down packing prepares an R-tree from root-level to leaf-level by splitting. These R-tree packing methods motivated our parallel R-tree construction on GPUs.

Some research has been done in the 90's for concurrent R-tree on multi-core architecture [27], [28] and on distributed systems [29]. The basic idea for concurrent R-tree on multi-core architecture is that with the help of concurrent control techniques such as lock coupling [28], linking [27] and retrying [30], certain insertion, deletion or search operations can be performed concurrently. Thus the construction of R-tree and its search operation can be accelerated. On the other hand, R-tree on distributed systems aims to improve R-tree's scalability by de-clustering the R-tree nodes on independent computer nodes. In [18], the authors present algorithms for concurrent R-tree construction and query on GPUs. However, one of the limitations in their data structure is the allocation of a fixed amount of memory for each R-tree node, thus causing a waste of device's global memory and subsequent loss of performance. Moreover, the resulting quality of R-Tree is poor, leading them to prefer a sequentially-built R-tree for parallel query.

B. Parallel R-tree Query

A parallel R-tree query (but not construction) algorithm is reported in [1]. However, the algorithm does not fully

explore the parallelism in R-tree query, and its underlying assumption that the entire R-tree can be loaded into GPU's shared memory, which is shared by only 8-16 cores of an SM and is much smaller than its device memory, is not realistic. The breadth first search (BFS) based parallel R-tree query algorithm in [18] allocates device memory based on the type of application and sampling of a few queries. This can cause allocating of excessive memory for each query, and the system may result in memory overflow when processing large data sets. In an alternate algorithm to handle the memory overflow case, the query algorithm becomes complicated and performance suffers. [31] outlines a bulk loading approach to R-tree construction using simple linear arrays. Our approach is similar in that we also pre-compute the size of the tree in order to optimize the memory usage on the GPU, however our algorithm differs in that we use Geo-Packing principles to pre-sort the MBRs in order to minimize branching. [31] also outlines several search paradigms however as noted in the paper it suffers from additional overhead of using non-native data primitives.

III. NVIDIA GPUS AND CUDA ARCHITECTURE

Modern General-purpose graphics processing units (GPGPUs or GPUs) are fully programmable many-core graphic processing units. A GPU consists of an array of parallel processors which are often referred to as streaming multiprocessors (SM). For example, in nVIDIA's Tesla k20c which is based on their Kepler architecture (GK110) and has 13 SMs, a single SM consists of 32 scalar processors with each scalar processor equipped with large amounts of registers. Each SM also has 48 KB on-chip shared memory which has lower access latency and higher bandwidth compared to device memory which is accessible to every SM, and has larger size. The SMs employ a Single Instruction Multiple Thread (SIMT) architecture. A group of 32 threads called a warp is the minimum execution unit. Once scheduled on a SM, the threads in a warp share the same instruction and can execute in a fairly synchronous fashion.

Compute Unified Device Architecture (CUDA) is a parallel computing architecture developed for parallel processing and enables access to the instruction sets and memory in the nVIDIA GPUs. A typical CUDA program is organized into host programs and one or more parallel kernel programs. The host programs are executed on CPU while the parallel kernel programs run on GPUs.

Some performance considerations are worthy of more discussion. The first one is the number of threads spawned in a CUDA program. When the instructions executed by the threads in a warp must wait for a long-latency operation such as device memory read, other warps are scheduled to be executed. The mechanism of tolerating the latency of expensive operations using work from other warps is often referred to as latency hiding. The mechanism can work due to the fact that the thread management in GPUs is implemented in hardware and is extremely efficient. Therefore, a CUDA program should be designed to spawn as many threads as possible so that enough threads are available to hide the latency

of expensive operations. The second consideration is the on-chip shared memory provided by the SM. Considering that the shared memory is much faster than device memory but smaller (roughly 48 kilobytes per thread block), it is advantageous if a CUDA program can partition the data in device memory into pieces that can fit into the shared memory, load these pieces into the shared memory one by one, process them, and then write the results back to device memory.

Recently introduced by nVidia, "Dynamic Parallelism" allows the launching of child GPU kernels from parent GPU kernels [32]. This brings the ability to easily implement recursion and very fine-grained process forking on the GPU. This paradigm lets the GPU re-partition its workload on demand and allocate its limited resources to high compute demand tasks away from low compute demand tasks. It must be used carefully however as the overhead of launching GPU kernels is relatively high and it is important to ensure that child kernels launch with some minimum number of threads for the given task in order to offset this overhead.

IV. PARALLEL R-TREE CONSTRUCTION ON THE GPU

Our parallel R-tree construction is performed in a bottom-up fashion and has three steps:

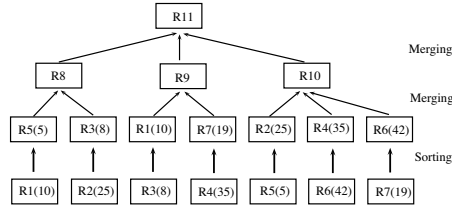
- Step 1: Map MBRs of polygons onto Z-order curve [33], and compute morton number [33] for each of the MBRs. Then sort these MBRs on their morton numbers (on GPU).
- Step 2: Determine resulting R-tree structure (on CPU).
- Step 3: Build the R-tree in a level-by-level fashion (on GPU).

Our primary consideration when building an R-tree is the quality of the resulting R-tree. As discussed in the previous section, for a better R-tree search performance, it is crucial to minimize the dead area and the overlap between MBRs. The basic idea in our R-tree construction is based on the Geo-Packing Principle that if R-tree can be built by grouping spatially neighboring MBRs, the dead area in resulting MBRs and overlap between these MBRs can be decreased. In mathematical analysis, a space-filling curve is a curve whose range contains a d -dimensional hypercube. A one-to-one relationship can be developed between every point in a space-filling curve and every hypercube in d -dimensional space. In other words, a space-filling curve can be used to "linearize" a d -dimensional space. Considering the locality property of the space-filling curve, we can determine the neighborhood of a d -dimensional hypercube by considering its corresponding point in the space-filling curve. If we consider a MBR as a d -dimensional point or a d -dimensional hypercube, the space-filling curve is well suited for identifying the neighboring MBRs. The calculation of the space-filling value for each spatial object's MBR is independent, and therefore can be easily parallelized. The technique for efficiently calculating a morton number is described in [34]. A modified version of radix sort for CUDA [35] can be used to sort the spatial objects.

In the sequential R-tree construction, two R-tree parameters, m , the minimum index entry size, and M , the maximum index entry size, are used to determine when to split a node for

insertion or merge nodes for deletion. These parameters are decided before an R-tree is built. Likewise, in our algorithm we need to choose m and M prior to construction of the R-tree. Once these parameters are decided, the resulting R-tree structure is also determined. For an R-tree is built on n MBRs, the maximum number of R-tree nodes at the leaf-level is $\lceil \frac{n}{m} \rceil$ since each node has at least m index entries. More generally, if level i of an R-tree has n' nodes, then level $i+1$ above has $\lceil \frac{n'}{m} \rceil$ nodes, with leaves at level 0. Once the resulting R-tree structure is known, the device memory needed to store the R-tree can be pre-allocated. Algorithm 1 sketches how to compute MBRs for each node, and how to develop relationships between parent nodes and child nodes. For the sake of implementation convenience, we change the way an R-tree node is traditionally represented. An R-tree in our algorithm is stored in a heap-like fashion. All the nodes at every level are stored consecutively in an array. All nodes, represented by their MBRs, are stored and indexed in arrays. (We actually use four arrays to store each coordinate in MBRs in order to encourage memory coalescing). Array *sizes* is used to store the number of nodes at each level. A prefix sum computation on *sizes* allows quick access to nodes at a specific level. The hierarchy in traditional R-tree is implemented with two other helper arrays: *start* and *end*. The k -th elements in *start* and *end* array denote the first and the last child node index of the k -th node. Line 10-17 in Algorithm 1 describes the process of computing arrays *start* and *end*. Given that level $i-1$ has $sizes[i-1]$ nodes and level i has $sizes[i]$ nodes, Line 10-11 are executed to divide nodes at level i into $sizes[i-1]$ partitions, and k -th partition of nodes at level i will become child nodes of the k -th nodes at level $i-1$. Figure 3 illustrates concurrent construction of an R-tree on seven MBRs.

Fig. 3. R-tree construction example. (Morton numbers shown within parentheses)



V. PARALLEL R-TREE QUERY ON THE GPU

Our algorithms aim at carrying out a batch of queries on the R-tree which would be generated by one or more related applications or a large number of users. The processing of parallel R-tree query on CPU can be described as follows: a concurrently built R-tree is maintained in main memory, and is also shared by CPU threads. These CPU threads perform queries simultaneously in a depth-first-search (DFS) fashion, and generate query results.

Compared to R-tree queries on CPU, the parallel R-tree query on GPU is more complicated. Leveraging the GPU architecture which already holds the R-tree built using algorithm in

Algorithm 1 Construct an R-tree level i

```

1: procedure CONSTRUCT_LEVEL( $i$ )  $\triangleright$  this procedure will
   be executed concurrently by GPU threads. Each thread is
   working on one R-tree node.
2:    $int$  sizes[maxLevels]  $\triangleright$  # of nodes at each level
3:    $int$  prefixSum[maxLevels]  $\triangleright$  prefix sum on sizes
   array
4:    $int$  rnodeSize  $\triangleright$  # of nodes in the R-tree
5:    $int$  r[rnodeSize + n]  $\triangleright$  # MBRs for R-tree nodes
   and spatial objects
6:    $int$  start[rnodeSize], end[rnodeSize]  $\triangleright$  # starting
   and ending indices of child nodes
7:    $int$  offsetU, offsetL  $\triangleright$  # of nodes above the
   current level and lower level
8:    $int$  posFirst, posLast  $\triangleright$  index of first child node
   and last child node at lower level

9:    $id \leftarrow$  the index of current thread
10:   $posFirst = sizes[i-1] * id \div sizes[i]$ 
11:   $posLast = sizes[i-1] * (id+1) \div sizes[i] - 1$ 
12:   $offsetU \leftarrow prefixSum[i]$ 
13:   $offsetL \leftarrow prefixSum[i-1]$ 
14:   $node \leftarrow (id + offsetU)$ 
15:   $start[node] \leftarrow (offsetL + posFirst)$ 
16:   $end[node] \leftarrow (offsetL + posLast)$ 
17:  Initiate  $r[node]$ 
18:  for  $j \leftarrow posFirst, posLast$  do
19:    Merge  $r[node]$  with  $r[j + offsetU]$ 
20:  end for
21: end procedure

```

Section IV (at least partially) can significantly speed up overall query times considering that a part of the penalty with working with the GPU is copying data to and from the CPU to the GPU device. Here we present five algorithms for performing these queries, all of which can be performed in asynchronous batches and/or spread across multiple GPU devices on the same or multiple compute nodes (see Section VII) to overcome the resource limitations of the GPU.

A. Naive and Shared Leaf Level Query Algorithm

This algorithm is very straightforward and leverages the fact that since our parallel R-tree, and certain residual meta-data, reside in global memory on the GPU we can leverage this to short-circuit sections of the tree to speed up bulk search times. In particular, we leverage the fact that the R-tree must be stored in array form and we know *a priori* exactly where each level of the tree begins and ends.

This allows us to make an assumption: We do not need to know the path of any given query through the tree. This gives several benefits but comes with a limitation. This limitation is that any application that requires the path data of a query through the tree to be preserved cannot use this algorithm. The benefits however may outweigh the cost:

- We can work with the strengths of the GPU hardware that benefit from low branching and high arithmetic

complexity.

- The logic of the search is simple and straightforward.
- We can easily launch, maintain, and scale to meet the demands of the current query.

We have two versions of these algorithms that are nearly identical. The only major difference being that the non-naive version has extra logic which allows for coalesced memory reads from global memory into shared memory. The naive memory Leaf Level query algorithm is sketched as Algorithm 2.

Algorithm 2 Leaf Level Query Algorithm

```

1: procedure LEAF_QUERY
2:    $query \leftarrow queryMBR[thread]$   $\triangleright$  Each thread gets a
   single query
3:   for  $i \leftarrow 0, numLeaf$  do
4:     intersect  $queryMBR$  with  $leafMBR[i]$ 
5:   end for
6: end procedure

```

B. Level by Level BFS Query Algorithm

The traditional R-tree query algorithm is operated in the depth first search (DFS) manner in which the route of the search algorithm is data dependent and hence sequential. In contrast, this query algorithm is based on breadth-first-search (BFS) in order to introduce parallelism. Memory space required for query output can vary a lot. For some queries, its bounding query rectangle might not overlap with any of R-tree nodes, thus no memory space is needed. Yet in the worse case, R-tree might return all of the records as result. This variance in memory requirement places a huge challenge on the design of parallel query algorithms. GPU resources such as device memory are limited and thus cannot be preallocated for the worst case. The intermediate memory space allocated as BFS continues from root level to leaf level also needs to be well managed. Dynamic memory allocation is the traditional way out of this problem but for GPUs the dynamic memory allocation process adds significant overhead. Other issues include output arrangement and efficient use of GPU's on-chip memory.

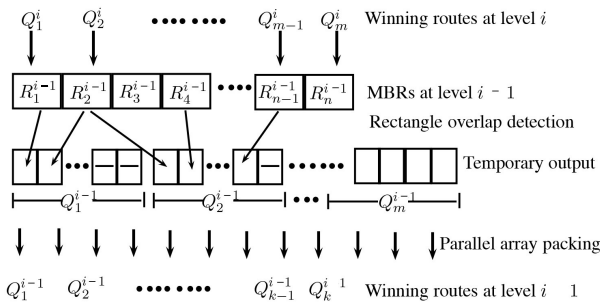


Fig. 4. R-tree parallel query on a level by level basis

The algorithm operates in a level-by-level fashion. As shown in Figure 4, multiple threads test the winning routes from level i ($Q_1^i, Q_2^i, Q_{m-1}^i, Q_m^i$) with MBRs at level $i-1$, and are in search of the winning routes that are yet to be traversed at the levels $i-1$. The resulting winning routes ($Q_1^{i-1}, Q_2^{i-1}, Q_{k-1}^{i-1}, Q_k^{i-1}$) at level $i-1$ are stored in the pre-allocated array, and gathered for the next level processing using array packing. The memory allocation at each level is exactly computed, and released after the algorithm travels to the next level. The query outputs are also packed in an array, making the follow-up processing convenient and efficient.

The efficiency of the algorithm is further improved as follows. On nVIDIA GPUs, the access to data in shared memory is at least an order of magnitude faster than that in device memory. For the upper levels of an R-tree, it is quite possible that an R-tree node will be accessed by many queries. So it is worth loading an R-tree node to shared memory, clustering the queries that will be performed on the same node and concurrently performing these queries. However, for the lower levels of an R-tree where the number of R-tree nodes become large, loading the R-tree node to shared memory does not necessarily improve search performance.

C. Dynamic Parallelism based DFS Query Algorithms

The dynamic parallelism based DFS query attempts to implement the sequential search algorithm on the GPU architecture without the massive branching and possible backtracking inherent in the sequential algorithm. It does this by leveraging nVidia's new dynamic parallelism paradigm which allows the launching of GPU kernels from GPU kernels. In general this allows further speedup by minimizing potential branching while also allowing the GPU to adapt to the problem by performing more computation where it is needed and less where it is not.

These properties make this new paradigm ideal for the purposes of traversing tree like data structures as it allows us to launch new kernels whenever we would have forked into a new branch (or branches) of the tree. This also completely removes the problem of having to backtrack when multiple, potentially valid, paths through the tree must be explored. Conceptually the idea is very simple and similar to recursion but we also have the option of launching new kernels in a non-blocking fashion.

The downside to this approach is the overhead of launching a kernel which can be rather high and the potential to launch many thousands of kernels for larger R-trees. The basic steps are sketched as Algorithm 3.

Modified Dynamic Parallelism based DFS query:

The shared leaf level algorithm with all k query searching over all leaves has $O(kn)$ growth therefore it is expected to be suitable only for small trees. The level-by-level BFS based algorithm has $O(k \log n)$ growth, but query branching can work against the CUDA's SIMT and coalesced memory access constraints and will incur array packing and other overheads. The DFS algorithm based on dynamic parallelism of CUDA

Algorithm 3 Dynamic Parallelism DFS Query Algorithm

```
1: procedure DYNDFS_QUERY
2:   if !query then
3:     query  $\leftarrow$  queryMBR[thread]
4:   end if
5:   if intersect query with nodeMBR[thread] AND
     nodeMBR[thread]  $\neq$  leafnode then
6:     launch dynDFS_Query() kernel
7:   end if
8:   if intersect query with nodeMBR[thread] AND
     nodeMBR[thread]  $\neq$  leafnode then
9:     output[]  $\leftarrow$  intersect query with
     nodeMBR[thread]
10:  end if
11: end procedure
```

will grow exponentially with a batch of queries fanning out to most of the tree. For practical reasons, hybrid approaches can have better performance. Our last algorithm is a blend of the best characteristics of the shared leaf level all to all kernel from Algorithm 2 with the dynamic parallelism based kernel of Algorithm 3 in order to reduce the unnecessary computation performed by the leaf level kernel and reduce the potential overhead incurred by the dynamic parallelism DFS kernel. It employs all-to-all search on the parents of the leaves which are a fraction of n and then launches kernels dynamically for selected set of leaves. The steps are sketched as Algorithm 4. As experiments will show, this algorithm is currently the best performer. Clearly starting at a higher level will further reduce all-to-all complexity but will run into overheads of larger subtree searches via dynamic kernels. We plan to explore the optimal starting point at a higher level.

Algorithm 4 Modified Dynamic Parallelism based DFS Query Algorithm

```
1: procedure MODDFS_QUERY1
2:   query  $\leftarrow$  leafParentMBR[thread]
3:   for  $i \leftarrow 0, numLeaf$  do
4:     trimmed_queryMBR[]  $\leftarrow$  intersect queryMBR
     with leafParentMBR[i]
5:   end for
6:   launch kernel:
7:   modDFS_Query2(trimmed_queryMBR[])
8: end procedure
9: procedure MODDFS_QUERY2(trimmed_queryMBR[])
10:  query  $\leftarrow$  trimmed_queryMBR[thread]
11:   $\triangleright$  Each thread gets a single query
12:  for  $i \leftarrow 0, numLeaf$  do
13:    intersect queryMBR with leafMBR[i]
14:  end for
15: end procedure
```

VI. EXPERIMENTS AND EVALUATION

A. Hardware and Data Environment

Our system used for test purposes is a Linux Infiniband cluster with heterogeneous nodes composed of multicore CPUs and several different types of nVidia GPUs. The main performance data was collected on a single core of an Intel Xeon CPU E5-2650 v2 running at a clock speed of 2.6 GHz with 64GB of main memory and a single nVidia Tesla K20c. For comparison reasons a standalone workstation with a single consumer grade nVidia GTX 970 was used to experimentally demonstrate how slight differences in GPU hardware can skew performance. The nVidia Tesla K20c is equipped with 2496 cores clocked at 705MHz and 5GB of DDR5 memory with 208 GB/s of bandwidth. By way of comparison, the nVidia GTX970 only has 1664 cores clocked at 1178MHz and 4GB of DDR5 memory with 224 GB/s of bandwidth². Some experiments were conducted on the on 448 cores Tesla C2075 compared with 1 CPU core of Intel Xeon E5-2620, and have been pointed out.

Data transfer time to and from the GPU is an important consideration but there are many variables to consider when accounting for it such as PCIe bus bandwidths, Disk I/O, etc. [37]. CPU Timing data here represents pure compute, and in the case of GPU, compute with transfer time across the PCIe bus, and does not include file I/O from disk. The timing of R-tree construction is compared both with and without copying all relevant data from the host CPU machine to the GPU memory. This allows us to more fairly compare the merits of the different architectures for construction. Once construction is completed, the R-tree resides on the GPU, so there is nothing further to copy other than the data being queried on the R-tree. Transfer time of the query data is included in both the timing plots and the speedup plots due to the effect of transfer time being very small but noticeable. However, due to the very small impact this time has on the speedup of the query, it is not separated out as with the construction.

The data files for evaluating parallel R-tree construction and query are extracted from the publicly available New York City Taxi Trip Data³. This data set is roughly 140 million rows of taxi trip data in New York City for the year of 2013 and is 50GB in size. Each row of the data set describes the start and end events of a given trip as well as some auxiliary data about the trip itself such as the number of passengers, the vendor, etc; each event has a discrete time and a location. These event locations are used in the following construction and query experiments to generate the initial MBRs. Size of the MBRs can be varied by adjusting the significant digits of an event's location that are used when constructing it's parent MBR. For experimental purposes this size was set to correspond roughly a few meters in radius around an event.

B. R-Tree Construction

We first evaluate the performance of parallel R-tree construction. Figure 5 demonstrates that a significant performance

²For a more complete picture of architectural differences see [36]

³Publicly available at http://chriswhong.com/open-data/foil_nyc_taxi/ and <http://www.andresmh.com/nyctaxitrips/>

gain is achieved while building an R-tree on the GPU. Sequential R-tree construction implementation we compared with are from [38]. Figure 6 shows the speedup achieved over the sequential algorithm, demonstrating over 200 fold speedup for construction of R-tree with 1M MBRs.

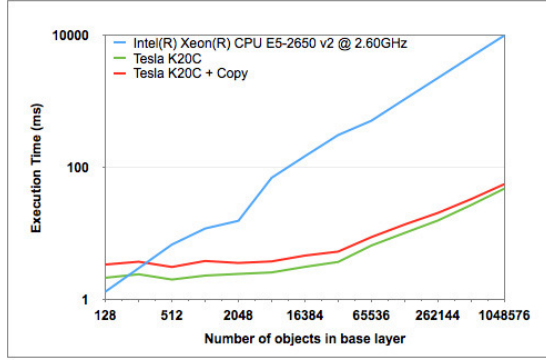


Fig. 5. Performance of Parallel R-tree construction (on Tesla K20C compared to sequential on 1 core Intel Xeon CPU, Taxi Data Set)

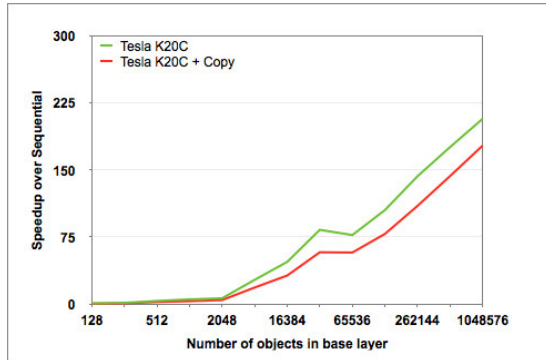


Fig. 6. Speedup of Parallel R-tree construction (on Tesla K20C compared to sequential construction on 1 core of Intel Xeon CPU, Taxi Data Set)

Parallel R-tree quality: The evaluation of parallel R-trees quality is not straightforward. As discussed earlier, a good R-tree has minimized dead area and MBR overlap. Unfortunately, these metrics are hard to measure and quantify. Therefore, hit counts on R-tree nodes when performing queries are employed as the metrics for our experiment. The basic construction algorithm starts with sorting of the MBRs. To assess the impact of sorting on quality, we also tried without sorting although the R-tree construction with MBRs initially sorted is less efficient, its resulting R-tree has much better quality, and therefore the performance of the queries on this R-tree is superior. Figure 7 shows the quality of the R-trees constructed, measured by hit counts per query on R-trees built on different numbers of MBRs. Our experimental procedure is as follows: the three R-trees are built with different construction algorithms on MBRs. Then a batch of queries using the same MBRs as query rectangles are performed on the resulting R-trees, and

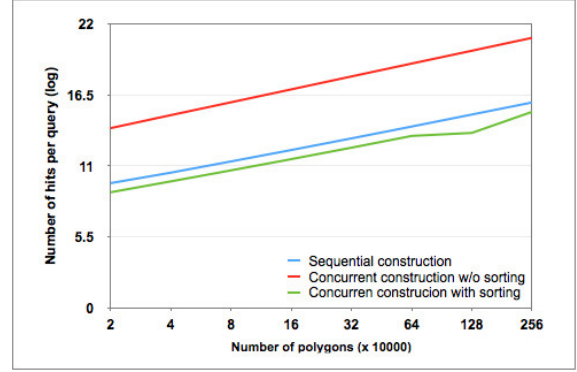


Fig. 7. Quality of Parallel R-Tree Constructed (Hit counts per query on R-trees built on different sizes of MBRs)

the frequency of R-tree nodes being accessed are counted. As figure shows, the quality of concurrently built R-tree (green line) is slightly better than sequential R-tree (blue line), but their difference is not as much as that between sequentially built R-tree and concurrently built R-tree without sorting.

In another experiment, we also compared the construction time for R-tree with and without sorting MBRs (Figure 8).

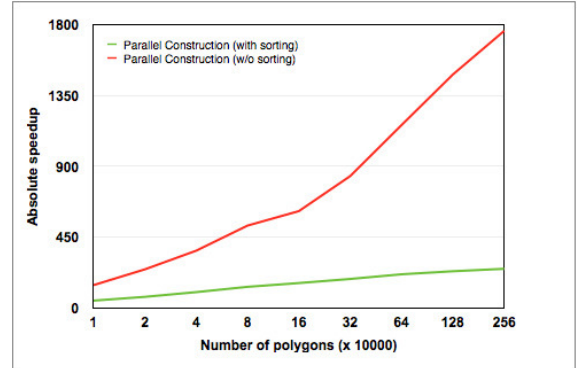


Fig. 8. Performance of Parallel R-tree construction (on Tesla C2075 compared to sequential on 1 CPU core of Intel Xeon CPU)

C. R-tree Query

The first experiment, Figures 9 and 10, was designed to compare each query algorithm while holding all possible parameters static. This necessitated that the R-Tree in this particular experiment be relatively small at only 16384 base layer objects so that a query of equal size to the R-tree itself could be performed. The GPU has adequate memory to construct large R-trees however in the worst case a base layer object could potentially intersect with every single other object in the tree. This necessitates wastefully allocating large areas of memory for search results that, in real world searches, are generally sparsely populated. On our Tesla K20c, which has 5GB of memory, we were only able to do a query of size 16K MBRs on this R-tree; the R-tree itself takes up roughly two

percent of this memory while the rest is filled by the query results. Figure 9 demonstrates that there is a significant speed up over sequential however each kernel behaves very different under different types of loads. The best performing algorithm was Algorithm 4 as it was able to most effectively utilize the limited resources of the GPU. Over all tests performed this kernel averaged a 80-fold speedup over sequential.

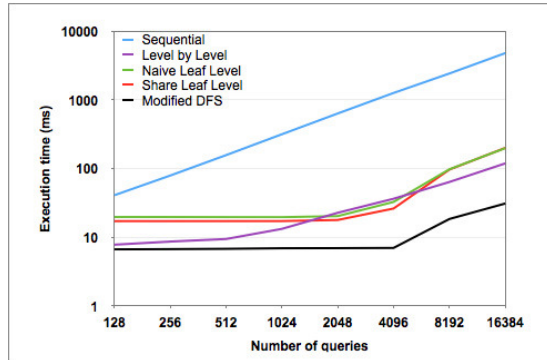


Fig. 9. Parallel R-tree query on Taxi Data, data transfer time to GPU included (on core Tesla K20C compared to sequential on 1 core Intel Xeon CPU)

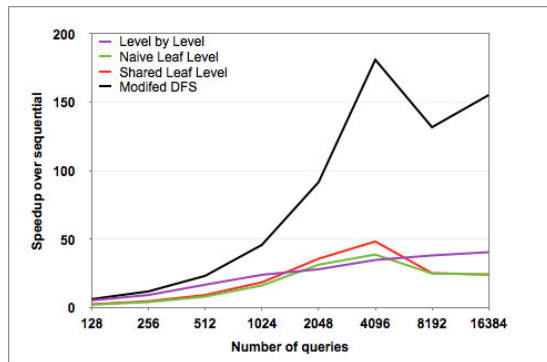


Fig. 10. Speedup of Parallel R-Tree query on Taxi Data, data transfer time to GPU included (on Tesla K20C Compared as compared to sequential construction on 1 core of Intel Xeon CPU)

The Dynamic Parallelism DFS based query was not experimented with as it was developed as a proof of concept and under anything but small queries and R-trees it performed very badly due to potentially launching exponential numbers of child kernels for each search. As such it is not discussed here.

The second experiment, Figures 11 and 12, was a simple iterative batch query designed to perform very large number of searches which otherwise cannot be accommodated on the GPU due to hardware memory limitations. Each large query is split into several smaller “batches” which are iteratively sent to the GPU in batches. In order to fully utilize the GPU hardware the largest size split was used. The Modified DFS batch query algorithm was left simple and un-optimized and as a comparison both the Shared and Naive leaf level batch

query algorithms were both moderately optimized to reduce the number of kernel launches. This was done to demonstrate and explore different performance options. Peak performance for a single query of size 4k in the previous experiment was 180-fold speedup. The average speedup for the batched iterative query with a batch size of 4k was 121-fold, with a range of 119-123 fold. Batched queries here incur overhead from launching kernels iteratively in addition to the added overhead of iterative copies to and from the GPU, however with tuning it should be possible to minimize the impact of this overhead.

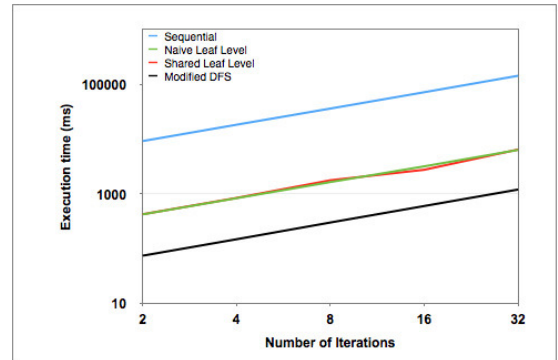


Fig. 11. Iterative batched Parallel R-tree query (on Tesla K20C compared to sequential on 1 core Intel Xeon CPU, R-tree size = 64k objects, batch size = 4k queries, Taxi Data Set, Logarithmic Scale)

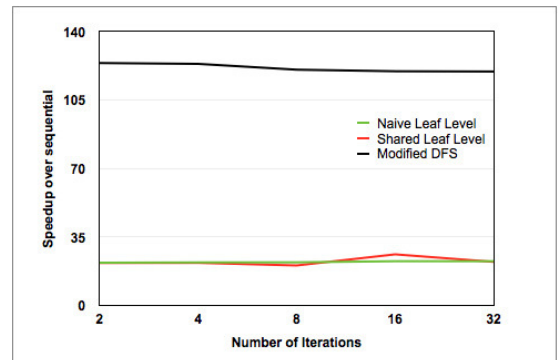


Fig. 12. Iterative batched query speedup (on Tesla K20C as compared to sequential on 1 core Intel Xeon CPU, R-tree size = 64k objects, batch size = 4k queries, Taxi Data Set)

The final experiment, Figure 13, sought to show how differences in GPU hardware, particularly in the Kepler architecture of the Tesla K20c and Maxwell architecture of the GTX970. The nature of the workstation housing the GTX970 necessitated using a small R-tree size in order to perform large number of queries. What is interesting here is that when run on the Tesla K20c there is very little difference between the Naive and Shared Leaf Level kernel speeds however on the GTX970 there is a more noticeable difference between the two before they converge. Another noticeable difference is in the general shape of the curves for Level by Level kernel which suggests that

the GTX970 is more sensitive to the moderate optimization and multiple kernel launches taking place in that particular algorithm.

VII. CONCLUSIONS AND FUTURE WORK

In this work, we developed and implemented a very efficient algorithm for R-tree construction giving over 200 fold speedup over sequential construction on state of the art hardware. We also developed and implemented several different search algorithms for R-tree on the GPU the best of which gives an average of 64 fold speedup over sequential. Currently the bottleneck for further experimentation is the resource starvation of the hardware itself.

Our roadmap for the future has us exploring asynchronous kernel launching and using multiple GPUs to perform the query. Also of interest is distributing the R-tree across multiple compute nodes with nVidia's new GPUDirect [39] technology which allows direct device-to-device addressing of GPU memory across the PCIe bus without having to first copy to host memory. The difference noticed in the final experiment illustrates that chip behavior can potentially be leveraged and algorithms tuned in order to achieve greater performance across our heterogeneous cluster that houses many different types of GPUs with very different feature sets and hardware specifications.

REFERENCES

- [1] M. Kunjir and A. Manthramurthy, "Using Graphics Processing in Spatial Indexing Algorithms," *Research report, Indian Institute of Science, Database Systems Lab*, 2009.
- [2] K. Zhou, Q. Hou, R. Wang, and B. Guo, "Real-time kd-tree construction on graphics hardware," in *ACM Transactions on Graphics (TOG)*, vol. 27, no. 5. ACM, 2008, p. 126.
- [3] Q. Hou, X. Sun, K. Zhou, C. Lauterbach, and D. Manocha, "Memory-scalable gpu spatial hierarchy construction," *IEEE Transactions on Visualization and Computer Graphics*, 2010.
- [4] S. Popov, J. Günther, H. Seidel, and P. Slusallek, "Stackless kd-tree traversal for high performance gpu ray tracing," in *Computer Graphics Forum*, vol. 26, no. 3. Wiley Online Library, 2007, pp. 415–424.
- [5] D. Benson and J. Davis, "Octree textures," in *ACM Transactions on Graphics (TOG)*, vol. 21, no. 3. ACM, 2002, pp. 785–790.
- [6] G. Ziegler, R. Dimitrov, C. Theobalt, and H. Seidel, "Real-time quadtree analysis using HistoPyramids," *Real-Time Image Processing 2007*, vol. 6496, 2007.
- [7] P. Ajmera, R. Goradia, S. Chandran, and S. Aluru, "Fast, Parallel, GPU-based Space Filling Curves and Octrees," *ISD2008*.
- [8] R. Castro, T. Lewiner, H. Lopes, G. Tavares, and A. Bordignon, "Statistical optimization of octree searches," in *Computer Graphics Forum*, vol. 27, no. 6. Wiley Online Library, 2008, pp. 1557–1566.
- [9] X. Sun, K. Zhou, E. Stollnitz, J. Shi, and B. Guo, "Interactive relighting of dynamic refractive objects," in *ACM SIGGRAPH 2008 papers*. ACM, 2008, pp. 1–9.
- [10] S. Lefebvre, S. Hornus, and F. Neyret, "Octree textures on the GPU," *GPU gems*, vol. 2, pp. 595–613, 2005.
- [11] K. Zhou, M. Gong, X. Huang, and B. Guo, "Data-parallel octrees for surface reconstruction," *IEEE Transactions on Visualization and Computer Graphics*, 2010.
- [12] H. Grahnl, N. Lavesson, M. Lapajne, and D. Slat, "A CUDA Implementation of Random Forests-Early Results," in *Third Swedish Workshop on Multi-core Computing*. Chalmers Institute of Technology, 2010.
- [13] T. Sharp, "Implementing decision trees and forests on a GPU," *Computer Vision-ECCV 2008*, pp. 595–608, 2008.
- [14] C. Lauterbach, M. Garland, S. Sengupta, D. Luebke, and D. Manocha, "Fast BVH construction on GPUs," in *Computer Graphics Forum*, vol. 28, no. 2. Wiley Online Library, 2009, pp. 375–384.
- [15] S. Lefebvre and H. Hoppe, "Perfect spatial hashing," in *ACM SIGGRAPH 2006 Papers*. ACM, 2006, pp. 579–588.
- [16] D. Alcantara, A. Sharf, F. Abbasinejad, S. Sengupta, M. Mitzenmacher, J. Owens, and N. Amenta, "Real-time parallel hashing on the GPU," *ACM Transactions on Graphics (TOG)*, vol. 28, no. 5, pp. 1–9, 2009.
- [17] R. Pagh and F. Rodler, "Cuckoo hashing," *Algorithms-ESA 2001*, pp. 121–133, 2001.
- [18] L. Luo, M. D. F. Wong, and L. Leong, "Parallel implementation of R-trees on the GPU," in *ASP-DAC*, 2012, pp. 353–358.
- [19] S. K. Prasad, S. Shekhar, M. McDermott, X. Zhou, M. Evans, and S. Puri, "GPGPU-accelerated Interesting Interval Discovery and Other Computations on GeoSpatial Datasets: A Summary of Results," in *Proceedings of the 2Nd ACM SIGSPATIAL International Workshop on Analytics for Big Geospatial Data*, ser. BigSpatial '13. New York, NY, USA: ACM, 2013, pp. 65–72. [Online]. Available: <http://doi.acm.org/10.1145/2534921.2535837>
- [20] A. Guttman, "R-Trees: A Dynamic Index Structure for Spatial Searching," in *SIGMOD Conference*, 1984, pp. 47–57.
- [21] R. Bayer and E. M. McCreight, "Organization and maintenance of large ordered indices," *Acta Inf.*, vol. 1, pp. 173–189, 1972.
- [22] N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, "The R*-Tree: An Efficient and Robust Access Method for Points and Rectangles," in *SIGMOD Conference*, 1990, pp. 322–331.
- [23] I. Kamel and C. Faloutsos, "Hilbert R-tree: An Improved R-tree using Fractals," in *VLDB*, 1994, pp. 500–509.
- [24] S. Leutenegger, M. Lopez, and J. Edgington, "STR: A simple and efficient algorithm for R-tree packing," in *Data Engineering, 1997. Proceedings. 13th International Conference on*. IEEE, 1997, pp. 497–506.
- [25] L. Chen, R. Choubey, and E. A. Rundensteiner, "Merging R-Trees: Efficient Strategies for Local Bulk Insertion," *GeoInformatica*, vol. 6, no. 1, pp. 7–34, 2002.
- [26] Y. J. García, M. A. Lopez, and S. T. Leutenegger, "A Greedy Algorithm for Bulk Loading R-Trees," in *ACM-GIS*, 1998, pp. 163–164.
- [27] V. Ng and T. Kameda, "The r-link tree: A recoverable index structure for spatial data," in *Database and Expert Systems Applications*. Springer, 1994, pp. 163–172.
- [28] J. K. Chen, Y.-F. Huang, and Y.-H. Chin, "A Study of Concurrent Operations on R-Trees," *Inf. Sci.*, vol. 98, no. 1–4, pp. 263–300, 1997.
- [29] B. Schnitzer and S. T. Leutenegger, "Master-Client R-Trees: A New Parallel R-Tree Architecture," in *SSDBM*, 1999, pp. 68–77.
- [30] A. Biliris, "Operation-specific locking in balanced structures," *Inf. Sci.*, vol. 48, no. 1, pp. 27–51, 1989.
- [31] S. You, J. Zhang, and L. Gruenwald, "Parallel spatial query processing on gpus using r-trees," in *Proceedings of the 2Nd ACM SIGSPATIAL International Workshop on Analytics for Big Geospatial Data*, ser. BigSpatial '13. New York, NY, USA: ACM, 2013, pp. 23–31. [Online]. Available: <http://doi.acm.org/10.1145/2534921.2534949>
- [32] nVidia. (2012) http://developer.download.nvidia.com/assets/cuda/docs/TechBrief_Dynamic_Parallelism_in_CUDA_v2.pdf.
- [33] G. Morton, "A computer oriented geodetic data base and a new technique in file sequencing," *Tech. Rep.*, 1966.
- [34] "Interleave bits by Binary Magic Numbers," Website, 1997. [Online]. Available: <http://graphics.stanford.edu/~seander/bithacks.html#InterleaveBMN>
- [35] N. Satish, M. Harris, and M. Garland, "Designing efficient sorting algorithms for manycore GPUs," in *Parallel & Distributed Processing, 2009. IPDPS 2009. IEEE International Symposium on*. IEEE, 2009, pp. 1–10.

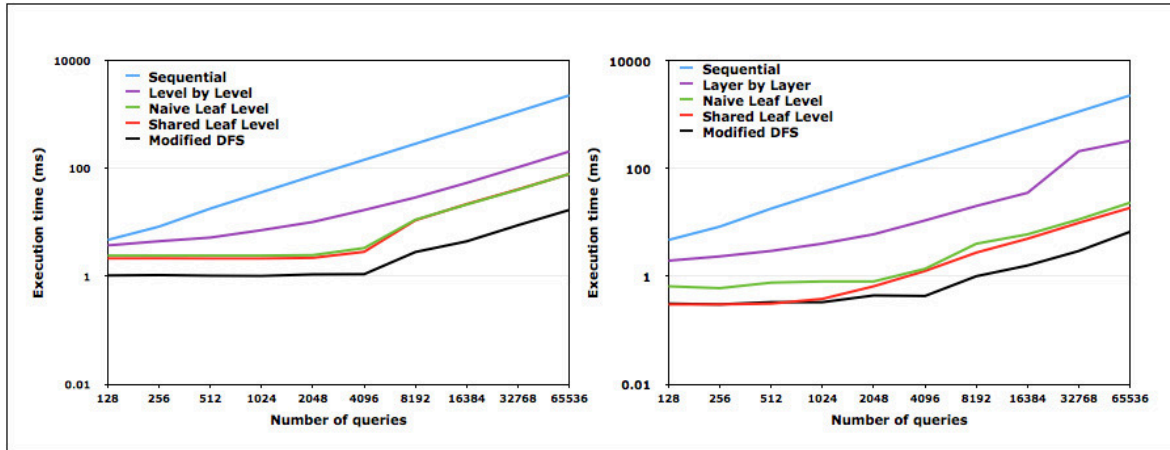


Fig. 13. Hardware Comparison for Parallel R-tree query (Tesla K20C on left, GTX970 right as compared to to sequential on 1 core Intel Xeon CPU)

- [36] nVidia, <http://devblogs.nvidia.com/parallelforall/5-things-you-should-know-about-new-maxwell-gpu-architecture/>, 2014.
- [37] C. Gregg and K. Hazelwood, "Where is the data? why you cannot debate cpu vs. gpu performance without the answer," in *Performance Analysis of Systems and Software (ISPASS), 2011 IEEE International Symposium on*. IEEE, 2011, pp. 134–144.
- [38] "R-tree Implementation," Website, 2013. [Online]. Available: <http://www.superliminal.com/sources/sources.htm>
- [39] nVidia. (2014) <http://docs.nvidia.com/cuda/gpudirect-rdma/>.